

Plataforma .NET

Introducción

Arquitectura del Software

Curso 2025/26

Introducción

.NET es una plataforma de desarrollo gratuita, de código abierto y multiplataforma.

- Microsoft incentiva las contribuciones por parte de la comunidad de desarrolladores de .NET que no forman parte de Microsoft
- .NET funciona tanto en Windows como Linux o macOS.
- Ofrece una extensa biblioteca de código bien documentada, con guías y tutoriales de programación.

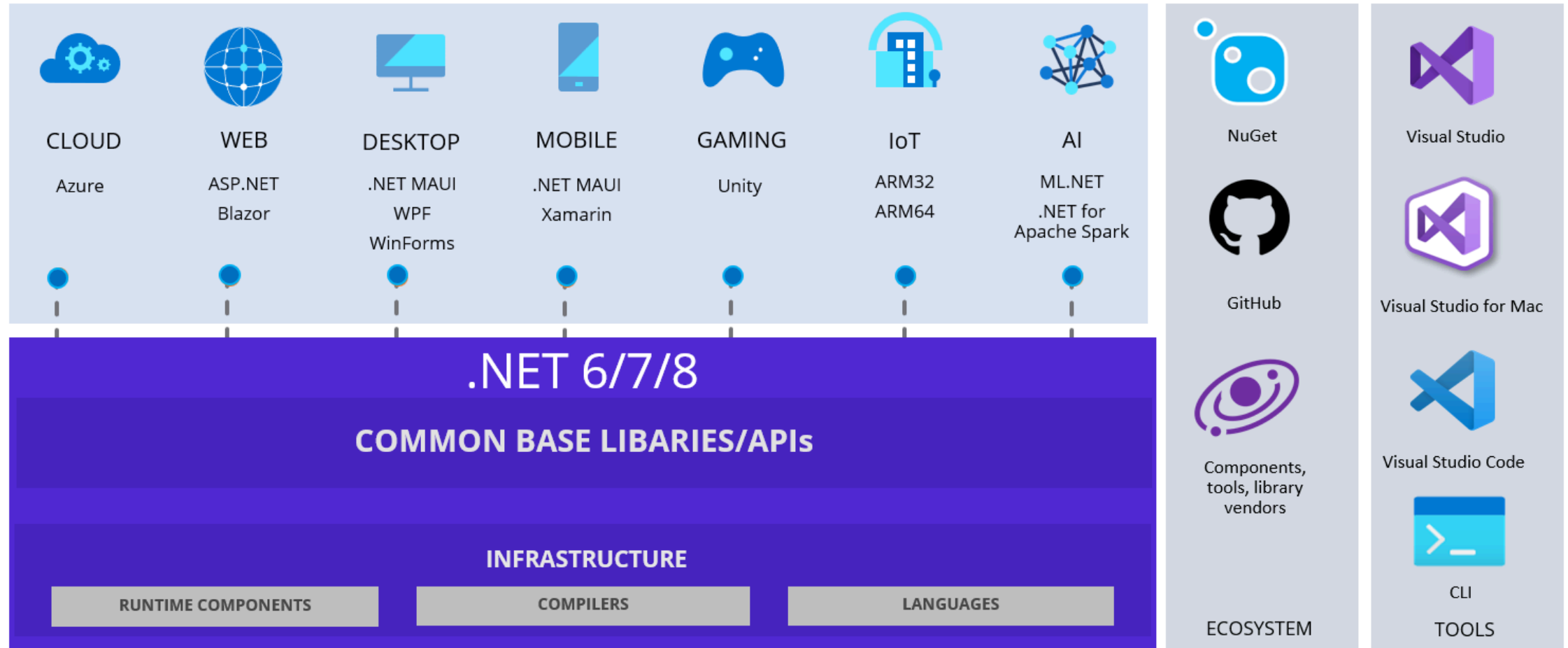
Breve historia de .NET

- La versión original de .NET Framework se lanzó por primera vez a principios de 2002.
- .NET Framework no era multiplataforma, así que Microsoft empezó a trabajar para que los desarrolladores de .NET pudieran trabajar en macOS, Linux y Windows.
- En 2014 se presentó .NET Core. Microsoft ha mantenido la versión original de .NET Framework, pero las nuevas características y mejoras se reservan para .NET Core.
- Más adelante, se suprimió Core en el nombre. Las siguientes versiones principales son .NET 5, .NET 6, .NET 7, .NET 8, etc.

.NET vs .NET Framework

.NET	.NET Framework
Se ejecuta en Linux, macOS y Windows	Solo se ejecuta en Windows
Código abierto y acepta contribuciones de la comunidad	El código fuente está disponible, pero no acepta contribuciones
Soporta más tipos de aplicaciones y ofrece un mejor rendimiento	Solo recibe correcciones de errores de seguridad y confiabilidad
No se incluye con el sistema operativo	Incluido en Windows y actualizado a través de las actualizaciones de Windows

Ecosistema .NET



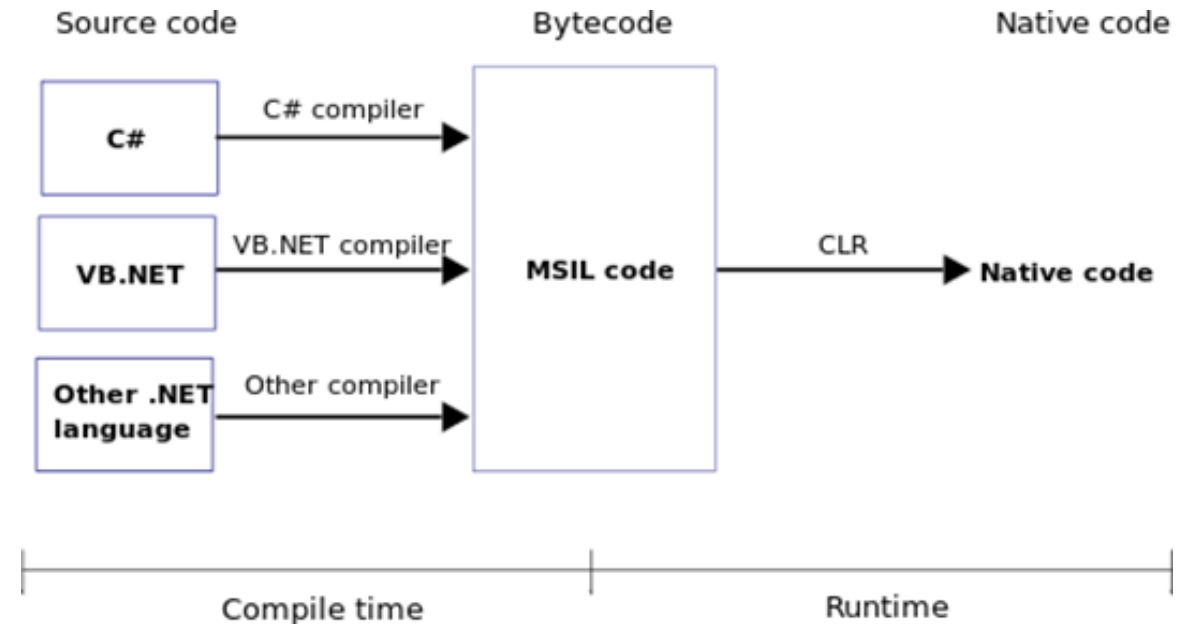
Soporte multilinguaje

El aspecto más destacable de .NET fue el **soporte multilinguaje**:

- El código del lenguaje de programación compila a código intermedio *bytecode* (llamado *Microsoft Intermediate Language*, MSIL o *Code Intermediate Language* CIL) que ejecuta el entorno de ejecución.
- El compilador de .NET guarda el código IL en un archivo denominado *ensamblado*.

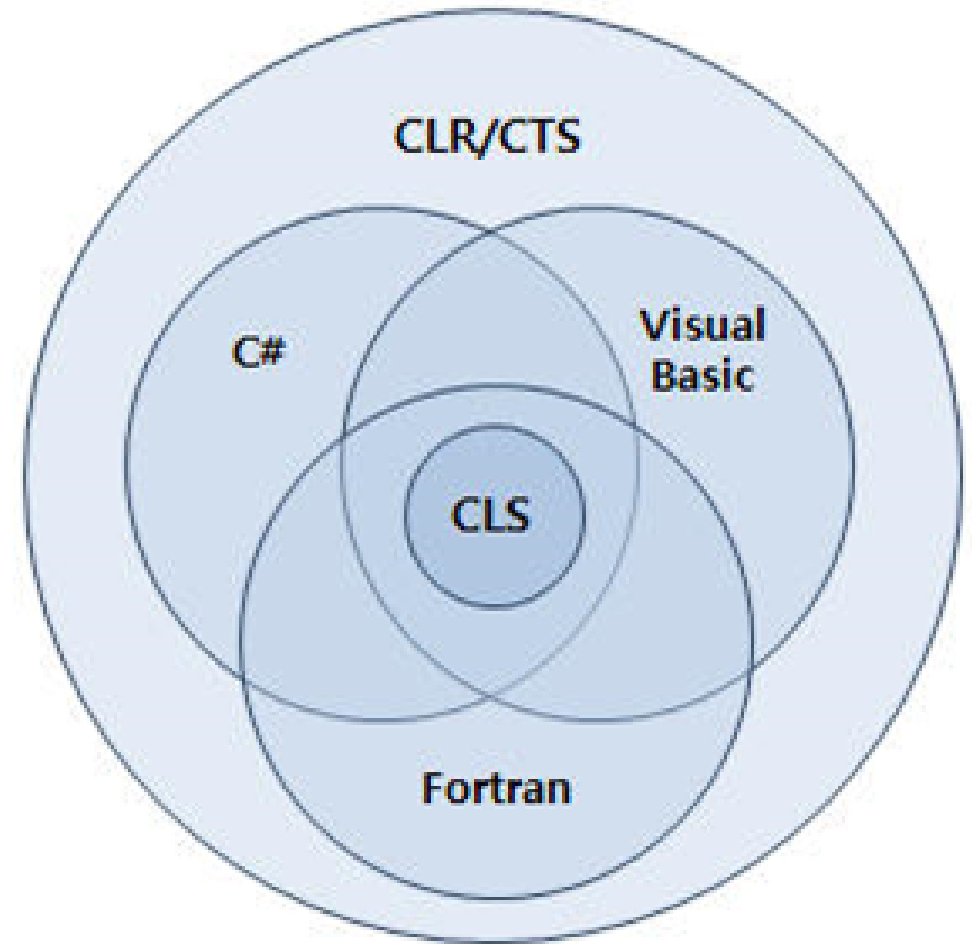
Entorno de ejecución - CLR

- El código intermedio es ejecutado utilizando una *máquina virtual* denominada CLR (*Common Language Runtime*).
- Esta máquina virtual NO interpreta el código, sino que siempre lo compila a código nativo antes de su ejecución.



Sistema común de tipos

- El entorno de ejecución ofrece un **sistema común de tipos** (*Common Type Subsystem*, CTS): conjunto de reglas que sigue el CLR al declarar, usar y administrar tipos.
- Permite la integración entre lenguajes y la seguridad de tipos.



Sistema común de tipos

- Clases y estructuras usadas para representar los tipos de datos.
- CTS declara los diferentes tipos de datos, junto con la especificación de cómo se gestionan en tiempo de ejecución y la seguridad de tipos.
- *Common Language Specification* (CLS): Subconjunto de CTS. Define las reglas que debe seguir todo lenguaje que se ejecuta en .NET, garantizando que los programas desarrollados en diferentes lenguajes de .NET puedan interactuar entre sí.

Lenguaje C#

- Si bien .NET es un entorno multilenguaje, Microsoft creó el **lenguaje C#** para aprovechar el sistema de tipos de la plataforma (CTS).
- Inspirado especialmente en Java y C++, tiene una sintaxis similar, por lo que es fácil de aprender si vienes de esos lenguajes.
- El [ranking TIOBE para C#](#) es un buen indicador de la aceptación de la plataforma .NET.
- [Documentación del lenguaje](#)

C#

- Tipado estático y fuerte: detecta errores en tiempo de compilación.
- Manejo de memoria con Garbage Collector: reduce fugas de memoria.
- Programación Orientada a Objetos: soporta encapsulación, herencia y polimorfismo.
- Eventos y Delegados: implementación del patrón Observer.
- Manejo de excepciones: bloques try-catch-finally.
- LINQ (Language Integrated Query): Permite consultas sobre colecciones de datos.
- Asincronía con async/await: facilita programación asíncrona.

Propiedades en C#

- Una propiedad en C# es una definición de variable y método (izda. imagen).
- C# permite definir propiedades implementadas automáticamente. Se usa por norma general en lugar de getters/setters explícitos (drcha. imagen).

```
public class Person{  
    private string firstName;  
    public string FirstName {  
        get { return firstName; }  
        set { firstName = value; }  
    }  
}
```

```
public class Person{  
    public string FirstName { get; set; }  
}
```

- Un accesor set puede tener `init` como modificador de acceso, haciendo el campo inmutable.

Código asíncrono en C#

- `async/await` es la forma estándar en C# de escribir código asíncrono sin bloquear el hilo.
- Se usa en ASP.NET Core porque permite manejar muchas peticiones al mismo tiempo de forma eficiente.

```
public async Task<string> GetDataAsync()  
{  
    var result = await httpClient.GetStringAsync("https://api.com/data");  
    return result;  
}
```

- `async` hace que el método devuelva un `Task`, que representa una operación asíncrona. `await` espera ese `Task` sin bloquear el hilo.

C# Nullable reference types

- En C#, las referencias podían ser `null` sin aviso. Esto provocaba errores en ejecución (*NullReferenceException*).
- Se introducen los **Nullable Reference Types**: El compilador avisa de posibles `null`. Fuerza a declarar explícitamente cuándo algo puede ser `null`.

```
string name;    // No debería ser null
string? name;   // Puede ser null
```

- No cambia el comportamiento de `null`. Solo añade comprobaciones del compilador

C# Records

- Los registros (modificador `record`) son un tipo especial de *clase* pensado principalmente para representar datos (más que comportamiento). Ofrece estas características:
 - Inmutable: las propiedades son init-only, y para modificar se usa with (que crea una copia).
 - Igualdad por valor: dos objetos son iguales si son del mismo tipo y almacenan los mismos valores.
- Sintaxis concisa: una sola línea basta para declarar constructor y propiedades.

C# Records

```
public record Person(string FirstName, string LastName)
{
    public string[] PhoneNumbers { get; init; }
}
public static void Main()
{
    Person person1 = new("Nancy", "Davolio") { PhoneNumbers = new string[1] };
    Console.WriteLine(person1);
    // output: Person { FirstName = Nancy, LastName = Davolio, PhoneNumbers = System.String[] }

    Person person2 = person1 with { FirstName = "John" };
    Console.WriteLine(person2);
    // output: Person { FirstName = John, LastName = Davolio, PhoneNumbers = System.String[] }
    Console.WriteLine(person1 == person2); // output: False

    person2 = person1 with { };
    Console.WriteLine(person1 == person2); // output: True
}
```

- Más sobre record

C#. LINQ

- LINQ (Language Integrated Query) permite hacer consultas directamente en C# sobre datos.
- Permite consultar colecciones (arrays, listas), bases de datos (con Entity Framework), XML, etc.

```
var lista = new List<int> { 1, 2, 3, 4, 5 };  
var pares = lista.Where(n => n % 2 == 0).ToList();
```

- [Ver LINQ](#)

C#. Espacios de Nombres (namespace)

Los namespaces en C# agrupan clases y evitan conflictos de nombres.

```
namespace MiAplicacion {  
    class Program {  
        static void Main() {  
            Console.WriteLine("Hola, mundo!");  
        }  
    }  
}
```

Similar a los paquetes en Java.

using para importar namespaces:

```
using System;  
using System.Collections.Generic;
```

ASP.NET Core

- Framework para construir aplicaciones web, APIs REST y microservicios sobre la plataforma .NET.
- Basado en un modelo de procesamiento de peticiones mediante un pipeline de middlewares configurable.
- Proporciona dos enfoques para crear API HTTP:
 - API mínimas. Recomendada en los nuevos proyectos.
 - API basadas en controladores.

ASP.NET Core

- Todo el código de inicio de la aplicación está en el archivo `Program.cs`.

```
var builder = WebApplication.CreateBuilder(args);
```

- Inicializa la aplicación, lee la configuración (`appsettings.json`), logging, entorno...
- Incluye inserción de dependencias integrada. Con `builder.Services` los servicios se agregan al contenedor de inyección de dependencias.

```
builder.Services.AddControllers();  
builder.Services.AddSingleton();  
builder.Services.AddScoped();
```

ASP.NET Core

- El pipeline (tubería de middleware) es la cadena de componentes por la que pasa cada petición HTTP antes de generar una respuesta.

```
app.UseMiddleware<ExceptionMiddleware>();  
app.MapActivitiesEndpoints();  
app.Run();
```

- En este ejemplo, primero se ejecuta un middleware de manejo de excepciones, y luego se enrutan las peticiones a los endpoints definidos.

API REST basada en Controlador

- Un controlador en ASP.NET Core es estructuralmente idéntico a un `@RestController` de Spring o un `@Path` en JAX-RS.

```
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    private readonly IProductService _service;

    public ProductsController(IProductService service)
    {
        _service = service; // Inyección por constructor
    }
    [HttpGet]
    public async Task<ActionResult<IEnumerable<ProductDto>>> GetAll()
        => Ok(await _service.GetAllAsync());
}
```

API REST mínima

- Permite crear puntos de conexión totalmente funcionales REST con código y configuración mínimos.

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<TodoDb>(opt => opt.UseInMemoryDatabase("TodoList"));
//...
var app = builder.Build();
app.MapGet("/todoitems", async (TodoDb db) =>
    await db.Todos.ToListAsync());
app.MapGet("/todoitems/complete", async (TodoDb db) =>
    await db.Todos.Where(t => t.IsComplete).ToListAsync());

app.Run();
```

API REST

Recursos:

- [Tutorial introductorio](#): Conceptos básicos para la creación de un API web basada en controlador que usa una base de datos.
- [Creación de API web con ASP.NET Core](#): amplía los conceptos del tutorial introductorio.
- [Creación de una API mínima con ASP.NET Core](#): tutorial para crear un API mínima.
- [Referencia rápida de las API mínimas](#): información adicional sobre las APIs mínimas.

OpenAPI

- OpenAPI es una especificación estándar para describir APIs REST.
- ASP.NET Core admite la generación de documentos openAPI en aplicaciones de API mínimas y basadas en controladores.
- Su uso ha cambiado a partir de la versión 9 de .NET.
- [Introducción a Swagger \(OpenAPI\)](#): Especificación independiente del lenguaje que sirve para describir API REST.
- [Información en .NET8](#)
- [Información en .NET10](#)

Entorno de desarrollo

- .NET SDK
 - En la asignatura se utilizará la versión 8.0 (LTS).
- Visual Studio Code:
 - Documentación del entorno
 - Incluimos la extensión C# de Microsoft (*C# for Visual Studio Code*).